

Completed-Shuffle Reuse Across Driver Attempts

Spark Project Improvement Proposal — discussion draft

Author: Unik Dahal

Discussion scope: Optional batch SQL recovery

Implementation baseline: ab0052497c0 in the current Spark fork

Status: Early community discussion; no JIRA assignment, shepherd or vote claimed

Q1. What are you trying to do?

Avoid repeating expensive work when a batch application's driver is replaced, if the output of that work still exists and the replacement independently establishes that it needs the same result.

More specifically, allow a replacement driver to adopt one complete retained SQL shuffle instead of running its producer map tasks again. The new driver plans its query normally. A connector certifies the actual source read; Spark constructs a bounded computation identity; a shuffle provider makes the old output readable. Only a compatible, complete candidate may suppress the new map stage. Otherwise Spark computes normally.

The decision requested

Should Spark support this optional semantic check and scheduler adoption mechanism for completed exchanges across driver attempts, building on the working prototype described here?

The request concerns the use case, ownership boundaries and integration direction. It does not ask the community to approve every private class, record format or provider protocol in the prototype. Those details are included in the companion so reviewers can assess feasibility against running code.

What already works

The prototype runs separate producer and replacement driver JVMs. A real source adapter and native shuffle provider exercise exact read certification, accepted-map publication, persistent discovery, local scheduler adoption and provider-native reads. GitHub Actions passed the original flow and two AQE modes: full reducers and actual reducer coalescing. Positive replacements launched zero producer map tasks; injected loss and expiry caused recomputation with correct results.

These are mechanism results from a small single-host fixture, not a claim of production readiness or a measured economic benefit. The feature is explicitly wired by the experimental harness; ordinary Spark queries do not automatically discover or reuse exchanges.

Reading guide: This proposal describes the product direction and risks. The [implementation and evidence companion](#) contains actual API excerpts, lifecycle diagrams, limits, test results and source links. Both describe the same implementation baseline.

Q2. What problem is this proposal NOT designed to solve?

It does not restore a SparkSession, driver heap or arbitrary application continuation. It does not select an older source snapshot, recover incomplete exchanges, checkpoint writes or streaming state, or replace remote shuffle storage. General cross-query caching and arbitrary result delivery are outside the initial discussion scope.

Initial discussion scope and demonstrated boundary

Area	Scope
Query	One selected batch SQL exchange with a certified V2 read and supported deterministic scan/filter/project producer. The harness selects the exchange explicitly.
Read identity	Actual planned scan and ordered mapper decomposition must match. Normal source planning errors remain errors. Changed source facts cause a miss.
Planning	No active runtime filtering or grouped scan partitions. No claim that every connector or AQE rewrite is supported.
Shuffle	Complete blocking row shuffle, compatible Spark revision and provider read format. Partial retained/fresh producer adoption is excluded.
Consumer evidence	JVM row collection in the native fixture. Arbitrary actions, writes, shared exchanges and downstream stage graphs need separate admission and recovery review.
AQE evidence	Full-reducer and contiguous coalesced-reducer reads passed. Skew, mapper-local, pipelined and push-merged adoption are not established.
Deployment	Trusted experimental deployment with explicit recovery group, manifest root and provider access. Production identity and authorization integration remain work.

These boundaries distinguish the tested flow from a general automatic SQL optimizer rule. The prototype has source/producer checks; it does not implement an exhaustive consumer-admission framework for arbitrary applications.

Q3. How is it done today, and what are the limits of current practice?

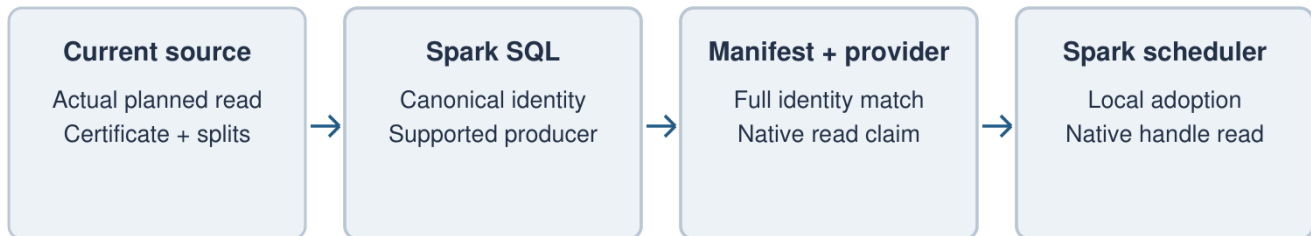
Spark can recompute lost shuffle output inside an application. External shuffle services, decommissioning and remote storage improve byte survival. A separately planned replacement driver still lacks proof that earlier bytes are the output of its own current exchange.

Explicit intermediate tables or files provide a durable application checkpoint today. They remain the more general solution, but require application changes and lifecycle choices. This proposal explores a transparent optimization at a narrower boundary.

[SPARK-25299](#) and [SPARK-54327](#) address remote shuffle storage. Their storage and integration direction should inform this work. The additional question here is semantic equivalence across driver attempts. `shuffleDataIO` provides sort-shuffle storage hooks; it does not itself certify SQL equivalence or decide whether a replacement dependency may skip map submission.

Q4. What is new in your approach and why do you think it will be successful?

The proposed addition is a Spark-owned proof-and-adoption boundary. It combines source certification, canonical producer identity, complete publication and a scheduler decision. Preserved bytes alone are insufficient.



Ownership and invariants

1. **The current read is authoritative.** Source resolution happens normally. Certification describes the same scan and partition objects the producer will execute; it cannot reconstruct an old read from the retained artifact.
2. **Spark owns computation identity.** Supported operators, expressions, output types, partitioning, source facts and compatibility settings enter a bounded encoding. Discovery compares the complete canonical payload, not just its lookup digest.
3. **The provider owns native bytes.** It seals the accepted map attempts and supplies an opaque descriptor and independent read lifecycle. Spark does not require the native provider to expose local index files.
4. **Spark owns map suppression.** Provider preparation precedes the local scheduler hook. Installation requires the current reservation, exact dependency and no existing map outputs. Otherwise ordinary execution wins.
5. **Failure invalidates the adopted exchange.** The native path fences its binding, clears retained tracker state and enters Spark's fetch-failure/recomputation machinery. It does not silently mix fresh producer outputs into a still-adopted generation.

Integration that exists today

The prototype uses an immutable filesystem manifest store and tracker-backed availability. Native reads are selected through the configured manager's wrapped shuffle handle. An opt-in preparation tag follows an exchange through AQE stage transformations; the final exchange is certified and prepared before map-stage submission.

This reuses existing query-stage materialization and scheduler recovery. Upstream review should assess whether these boundaries provide sufficient isolation and maintainability for the initial workload.

The source and provider contracts are generic. Iceberg and Celeborn are concrete integration examples used to test them, not requirements of the proposed Spark mechanism.

Q5. Who cares? If you are successful, what difference will it make?

Operators whose expensive batch applications are retried after driver loss could avoid repeating eligible producer work while its complete shuffle remains available. Savings depend on real driver-retry frequency, source stability, retention lifetime and how much of the retry cost precedes the exchange.

Verified mechanism evidence

The [native proof run](#) tested Spark commit `ab0052497c019d047fee2bea4b2dba1674ae5db4`. Each mode ran 13 independent driver invocations covering normal execution, publication, replacement, concurrent claims, misses and failures.

Replacement flow	Result rows	Producer map tasks	Native remote bytes	Actual AQE result
AQE disabled	32	0	730	Ordinary reader
AQE full reducers	32	0	730	Final adaptive plan
AQE coalesced	32	0	730	One merged reducer range

Every invocation produced the baseline result digest. Source changes and missing manifests recomputed. Artifact loss and lease expiry after adoption produced fetch failures, ran fresh producer work and returned the correct result. The [experimental CI run](#) also passed focused Core, SQL/AQE, source conformance and lint/license checks.

The fixture uses one host, `local[2]`, one source mapper and four reducers. Independent driver JVMs demonstrate the cross-driver boundary, but are not a multi-host deployment. The run proves native reuse and selected fallback behavior; it does not establish speedup, broad query eligibility or production security.

Evidence to establish value

Seek representative driver-retry incidents from independent operating environments. For each, report eligible producer cost as well as eligible query count, with ordinary AQE and source settings left visible. Include misses caused by changed reads, layout, unsupported expressions and expired retention.

Measure producer work avoided, discovery/preparation delay, first-attempt publication cost, retained storage, hit probability and failed-adoption cost. A single hit timing is not a workload-level benefit. No measured speedup or release threshold is claimed in this proposal.

If this bounded mechanism captures little real retry cost, prefer ordinary recomputation or explicit checkpoints. The working prototype makes the experiment possible; it does not predetermine that Spark should carry the maintenance cost.

Q6. What are the risks?

Risk	Current treatment and review needed
Incomplete semantic identity	Closed producer encoding and exact source binding reject unsupported cases. Connector truth remains a correctness assumption; source and expression coverage require review.
Shared dependency or consumer state	Availability is tracker-backed and dependency-scoped. General exchange reuse, simultaneous consumers and arbitrary downstream graphs are not established by the single-exchange fixture.
Loss after consumption starts	Existing fetch-failure and whole-stage retry machinery is used. Native tests inject faults after adoption, before reading; they do not prove every partial-result or side-effect boundary.
AQE statistics	Captured map sizes are scheduling estimates. Adopted SQL row count is unknown, avoiding a false empty-result inference. Broader statistics-driven rewrites require validation.
Access across applications	The PoC uses a trusted manifest root and provider control plane. Recovery-group strings are not authenticated retry credentials. Multi-tenant authorization and encryption need deployment design and tests.
Metadata and overhead	Bounded records and work queues constrain the prototype. Split metadata and map/reducer estimates remain size-limited; preparation may delay map submission. Large-scale cost is unmeasured.
Provider lifetime	Claims and renewal fence local reads; owner restart and expiry are tested. General durable revocation, known-bad artifact handling and service failover need review.

Compatibility and security boundary

Existing applications and connectors continue normally without experimental wiring. This is not a stable public API, and the proposal does not ask to add an obligatory method to Data Source V2 Batch or to stabilize the private shuffle-manager interface.

The identity includes the Spark build revision and provider format. Cross-version compatibility is not promised; build provenance and connector format discipline remain necessary. A digest is neither authorization nor confidentiality. The manifest can contain sensitive source metadata and must be protected along with retained data.

Current source access is resolved independently of reuse. That does not automatically authorize reading retained bytes from another application. Before production enablement, the configured provider and manifest store need a coherent access policy. The native fixture disables application I/O encryption and does not validate secure cross-attempt key management.

The initial product direction is opt-in recovery with ordinary recomputation on an unavailable or incompatible candidate. Normal source errors and Spark failures are not guaranteed to become successful retries. Side-effecting consumers must not be admitted on the strength of this PoC.

Q7. How long will it take?

The mechanism prototype and native AQE validation are complete for the recorded fixture. Upstream delivery still requires product feedback and Core, SQL and shuffle review. There is no committed release date or maintainer sponsorship.

The author will drive the discussion and follow-up implementation. Work should proceed in independently reviewable increments rather than merge the fork wholesale.

Increment	Deliverable
Community direction	Agree whether real retry cost warrants this mechanism and which initial workload should be supported. Identify maintainers and a PMC shepherd if proceeding formally.
Contract review	Review source identity, manifest trust, provider capability placement, scheduler installation and the supported consumer/AQE boundary against the current code.
Upstream implementation	Separate canonical identity/source binding, publication/discovery, native adoption and SQL wiring into reviewable changes with targeted tests.
Deployment validation	Exercise distributed executors, authorization/encryption, larger metadata, service failures and representative workloads before broader enablement.

Q8. What are the mid-term and final exams to check for success?

Mid-term: reproducible correctness and useful opportunity

A reviewer should be able to inspect one exact candidate, reproduce a retained hit and observe zero producer map tasks, correct results and native byte reads. Identity changes must miss. Adoption races and unavailable data must enter the intended fallback without accepting incompatible outputs. The recorded CI runs provide a starting point, not exhaustive conformance.

In parallel, establish cost-weighted opportunity from real retry incidents. Report the excluded queries and configurations. Do not tune the cohort until only the demonstration query remains.

Final: a maintainable, deployable improvement

Require an agreed consumer and source envelope, distributed failure coverage, reviewed access controls, bounded resource behavior and measured net benefit over recomputation. Confirm that disabled behavior remains unchanged and misses impose acceptable cost. Release decisions follow Spark's normal review and testing process.

The community can stop or narrow the proposal if benefit is weak or integration costs are disproportionate. Exact sample counts and performance thresholds belong in a prospective evaluation plan rather than the API contract.

Current review request

Feedback is most useful on workload demand, the tracker/handle adoption boundary, supported AQE readers, source certification and provider/manifest ownership. The next step is discussion, not a formal acceptance vote.

Appendix A. Proposed API changes

The current code uses private integration surfaces. Their names are useful anchors for review, not proposed stable public signatures.

Surface	Role
<code>ShuffleRecoverySourceBinding</code>	Associates source facts with the exact planned batch scan and ordered partitions.
<code>ShuffleRecoveryCertifiedBatchInputs</code>	Produces canonical identity inputs for the actual exchange.
<code>ShuffleRecoveryNativePublicationProvider</code>	Seals scheduler-accepted attempts into provider-native output.
<code>ShuffleRecoveryManifestStore</code>	Persists immutable records and finds earlier compatible candidates.
<code>ShuffleRecoveryNativeInstallation</code>	Exposes local install/invalidate operations and deferred provider cleanup.
<code>ShuffleRecoverySchedulerBackend</code>	Hooks adoption and invalidation into map selection and fetch-failure handling.
<code>ShuffleRecoveryExchangePreparation</code>	Prepares the final AQE exchange before map-stage submission.

For example, the implemented provider publication contract is:

```
trait ShuffleRecoveryNativePublicationProvider {
  def compatibilityId: String
  def seal(
    shuffleId: Int,
    acceptedAttempts: Vector[ShuffleRecoveryMapAttempt]): Vector[Byte]
}
```

Visibility annotations are omitted in this excerpt; the actual trait is `private[spark]`. The companion states preconditions and failure behavior. No new public `Batch` method or provider-owned discovery service is required by this draft.

Appendix B. Design sketch

Publication captures accepted map attempts, seals native output, rechecks the winner selection and publishes a manifest. Replacement preparation independently certifies the new exchange, compares full identity, obtains a provider claim and offers a local installation.

Before missing-map selection, the scheduler verifies the reservation and dependency, installs the native handle binding and replaces empty tracker availability. AQE uses the existing map-stage statistics path. On a native fetch failure, the binding and adopted tracker registration are invalidated, the epoch advances and a whole-stage retry marker feeds existing scheduler recovery.

The companion diagrams these transitions and identifies what is tested versus still requiring upstream review. The sketch and API excerpts refer to the tested code.

Appendix C. Alternatives considered

Alternative	Tradeoff
Always recompute	Simplest maintenance and no new persistent trust boundary. Prefer this when retained-hit opportunity is too small.
Explicit durable checkpoint tables/files	More general and application-controlled. Requires changes to the application and storage lifecycle; remains appropriate outside the narrow exchange scope.
Recover only inside the shuffle provider	Preserves native bytes but cannot independently establish current SQL semantics or decide whether Spark may omit a map stage.
Use query text, plan hash or snapshot ID alone	Insufficient: source read semantics, mapper layout, expression behavior, partitioning and format may differ.
Restore a driver/session	Much broader problem involving arbitrary state and side effects. Not necessary to test completed-exchange adoption.
Design a general semantic cache first	Expands sharing, policy and lifecycle obligations. Retry-driven reuse is the product motivation; strict authenticated lineage is not yet enforced by the PoC.
Replace the current scheduler path with a new retained-stage subsystem	Could offer different isolation, but its cost and need are unproven. Review the working tracker/handle approach first.

Questions for community discussion

- Which real driver-retry workloads have enough retained producer cost to justify this feature?
- Can the existing tracker-backed adoption and fetch-failure path be maintained with a clearly bounded consumer scope?
- Are source certification and provider publication the right extension boundaries, and how should manifest trust be integrated with deployment authorization?
- Which AQE and downstream shapes should be the first upstream target beyond the demonstrated full/coalesced reader flow?

References and package

- [Official SPIP process and template](#). This document follows its questions; no formal SPIP ticket or approval is implied.
- [SPARK-25299: remote shuffle storage](#) and [SPARK-54327: remote-storage proposal](#).
- [Current Spark fork at the tested implementation](#).
- [Native proof and artifacts; focused implementation CI](#).
- [Implementation and evidence companion](#): API contracts, source links, AQE and failure behavior, measured controls and remaining gaps.

The purpose of circulation is to obtain workload and maintainer feedback on the implemented direction. Production enablement and final interface choices remain subject to that review.